
Enterprise Infrastructure as Code, Resource Lifecycle, Rollbacks, Decommissioning, State Management & Production Operations

Cloud Architects

Platform Engineers

DevOps / SRE

AI Platform Engineers

100+ Pages • 18 Parts • Complete CLI Reference • Interview Questions
Troubleshooting Runbooks • Anti-Pattern Catalog • Production Checklists

Enterprise IaC Guide | 2025 Edition | OpenTofu Compatible

TABLE OF CONTENTS

Part 1 — Terraform Fundamentals & IaC Evolution

- › What is Infrastructure as Code?
- › IaC Tool Comparison Matrix
- › Declarative vs Imperative Paradigms

Part 2 — Terraform Architecture Deep Dive

- › CLI Lifecycle & Command Flow
- › Provider Plugin System
- › Resource Dependency Graph (DAG)

Part 3 — Resource Matching Internals

- › Resource Addressing & Identity
- › State Reconciliation Engine
- › Drift Detection & moved Blocks

Part 4 — State Management Deep Dive

- › Local vs Remote State
- › State Locking & DynamoDB
- › State Corruption Recovery

Part 5 — Complete CLI Command Mastery

- › Init, Validate, Plan, Apply, Destroy
- › State Sub-Commands Reference
- › Import, Console, Graph

Part 6 — Rollback Strategy & Failure Recovery

- › Git-Based Rollback
- › State-Based Recovery
- › Blue-Green & Canary Rollback

Part 7 — Infrastructure Decommissioning

- › Controlled Destroy Playbook
- › Partial Destroy & -target
- › Enterprise Decommission Framework

Part 8 — Force Destroy Deep Dive

- › force_destroy Patterns
- › terraform apply -replace
- › Data Loss Implications

Part 9 — State Manipulation Mastery

- › state mv / rm / import
- › moved Blocks (Modern)
- › Import Blocks (Terraform 1.5+)

Part 10 — Data Sources, Variables & Outputs

- › data vs resource
- › Variable Validation & Typing

- › Cross-Module Communication

Part 11 — Advanced Lifecycle Controls

- › create_before_destroy
- › prevent_destroy & ignore_changes
- › replace_triggered_by

Part 12 — Modules Mastery

- › Module Design & Versioning
- › Private Registries
- › Monorepo vs Multi-Repo

Part 13 — Enterprise Terraform Architecture

- › Environment & Workspace Strategy
- › GitOps Integration Patterns
- › Directory Structure Standards

Part 14 — Security Best Practices

- › Secret Management
- › IAM Least Privilege
- › State Encryption & Access Control

Part 15 — Anti-Pattern Catalog

- › 17 Critical Anti-Patterns
- › Consequences & Remediation

Part 16 — Troubleshooting Playbook

- › State Lock Resolution
- › Drift & Import Failures
- › Provider Upgrade Issues

Part 17 — System Retirement Programs

- › Data Center Exits
- › Cloud Migration Cleanup
- › Regulatory Decommissioning

Part 18 — OpenTofu & Future of Terraform

- › Licensing Changes & Migration
- › Feature Comparison
- › AI-Assisted IaC & Trends

Appendices: CLI Cheat Sheet • 125 Interview Questions • Production Readiness Checklist • Day-0/1/2 Operations Guide

Infrastructure as Code, Evolution & IaC Tool Comparison

1.1 What is Infrastructure as Code (IaC)?

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable definition files rather than through manual processes or interactive configuration tools. IaC treats infrastructure the same way software engineers treat application code — version-controlled, tested, peer-reviewed, and automatically deployed.

The core benefits of IaC are profound and transformational for engineering organizations:

- **Repeatability:** The same configuration produces identical infrastructure every time, eliminating snowflake servers and configuration drift.
- **Version Control:** Infrastructure changes are tracked in Git with full audit history, blame tracking, and the ability to revert.
- **Collaboration:** Teams can review infrastructure changes via pull requests using the same workflows as application code.
- **Speed:** Automated provisioning takes minutes instead of days of manual work.
- **Cost Control:** Infrastructure can be destroyed and recreated on demand, enabling ephemeral environments.
- **Disaster Recovery:** Entire environments can be rebuilt from code in minutes.

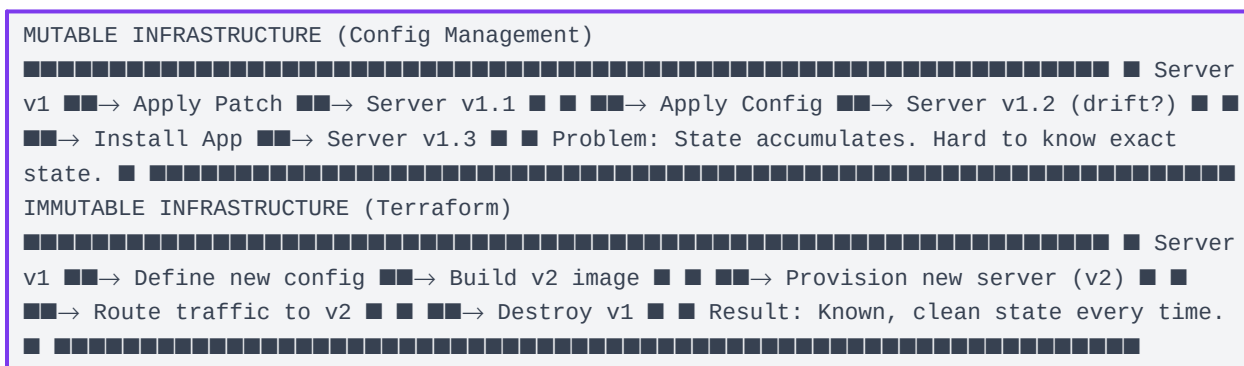
The Evolution of Infrastructure Management

Infrastructure management has evolved through distinct generations, each solving the problems of the previous era:

Era	Approach	Tools	Problems Solved	New Problems Introduced
Gen 1 ~2000s	Manual Provisioning	SSH, Web Console, CLI	Direct control	No repeatability, snowflakes, tribal knowledge
Gen 2 ~2008-2014	Config Management	Chef, Puppet, Ansible, SaltStack	Repeatable config, idempotent	Mutable infra, drift, ordering issues
Gen 3 ~2014-2019	Orchestration IaC	Terraform, CloudFormation	Immutable infra, declarative	State management complexity
Gen 4 ~2019-now	Platform Engineering	Backstage, Crossplane, Pulumi	Developer self-service	Abstraction overhead, learning curve
Gen 5 ~2024+	AI-Assisted IaC	Terraform + AI, OpenTofu	Natural language → infra	Hallucination risks, validation gaps

Immutable vs Mutable Infrastructure

A foundational concept that separates modern IaC from configuration management:



1.2 IaC Tool Comparison Matrix

Choosing the right IaC tool is a critical architectural decision. Here is a comprehensive comparison of all major tools:

Tool	Type	Language	State Mgmt	Multi-Cloud	Provider Ecosystem	Best For
Terraform	Declarative	HCL	tfstate file	■ Native	4000+ providers	Multi-cloud enterprise
OpenTofu	Declarative	HCL	tfstate file	■ Native	Terraform-compatible	Open-source Terraform
CloudFormation	Declarative	YAML/JSON	AWS managed	■ AWS only	AWS services only	AWS-native teams
AWS CDK	Imperative	Python/TS/Java	CloudFormation	■ AWS only	AWS via constructs	Developer-first AWS
Pulumi	Imperative	Python/TS/Go/C#	Pulumi Cloud	■ Native	100+ providers	Dev-heavy teams
Bicep	Declarative	Bicep DSL	Azure managed	■ Azure only	Azure services	Azure-native teams
Crossplane	Declarative	Kubernetes CRDs	Kubernetes	■ Native	Growing	K8s-native platform
Ansible	Imperative	YAML	None (stateless)	■ Via modules	Thousands	Config management

When to Choose Terraform

Terraform is the right choice when:

- Your infrastructure spans multiple cloud providers (AWS + Azure + GCP)
- You need to manage non-cloud resources (DNS, GitHub, Datadog, Snowflake, databases)
- Your team uses HCL and values the mature ecosystem of 4000+ providers
- You need strong state management with team collaboration via remote backends
- You require a strong module ecosystem with the Terraform Registry

■ Terraform is NOT the right choice when: you are 100% AWS-only and want deep CloudFormation integration, or your team strongly prefers imperative programming languages (consider Pulumi or CDK).

1.3 Declarative vs Imperative

This is one of the most important conceptual distinctions in infrastructure tooling. Understanding it deeply is essential to using Terraform effectively.

IMPERATIVE APPROACH (How to get there)

```

##### Step 1: Create VPC with CIDR
10.0.0.0/16 Step 2: Create subnet in us-east-1a Step 3: Create Internet Gateway Step 4:
Attach IGW to VPC Step 5: Create route table Step 6: Add route 0.0.0.0/0 → IGW Step 7:
Associate subnet with route table Step 8: Create security group with port 80 Step 9:
Launch EC2 t3.medium in subnet Problem: YOU must track current state and write every step.
Idempotency is YOUR responsibility. DECLARATIVE APPROACH (what the end state should be)

```

```

resource "aws_vpc"
"main" { cidr_block = "10.0.0.0/16" } resource "aws_instance" "web" { ami =
"ami-0abcdef1234567890" instance_type = "t3.medium" subnet_id = aws_subnet.main.id }
Result: Terraform figures out HOW to create/update/destroy. Idempotency is Terraform's
responsibility.

```

When you run `terraform apply`, Terraform performs a three-way diff:

- **Desired State:** What your .tf files declare
- **Known State:** What terraform.tfstate records from last apply
- **Actual State:** What currently exists in the cloud API (refreshed)

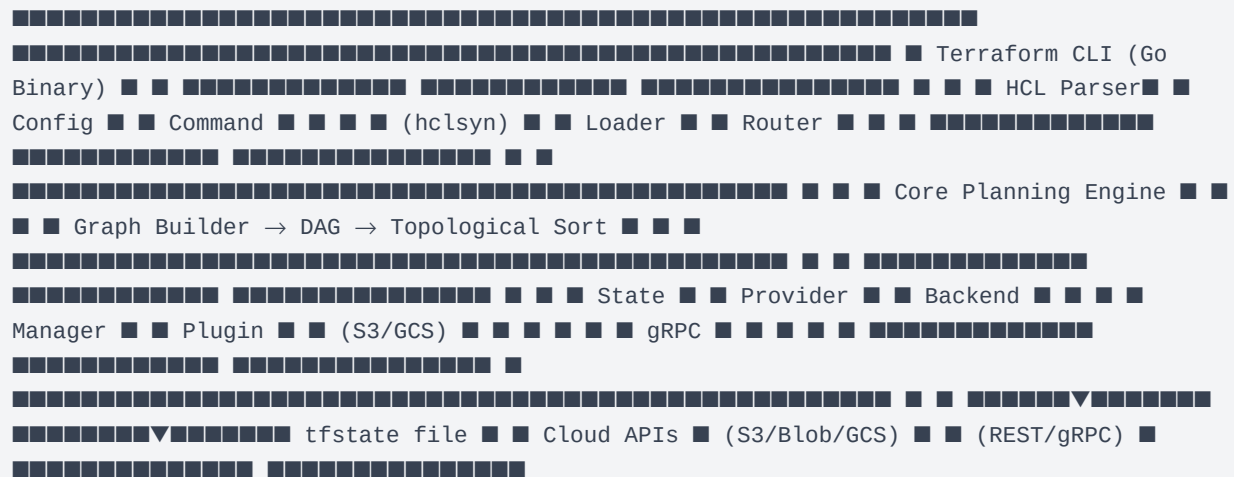
```
Terraform Reconciliation Loop ██████████ ■ 1. Parse HCL
→ Desired State ■ ██████████ ▼
██████████ ■ 2. Read tfstate → Known State ■
██████████ ■ ▼
██████████ ■ 3. Query Cloud APIs → Actual State■
██████████ ■ ▼
██████████ ■ 4. Compute Diff (Desired vs Actual)■
██████████ ■ ▼
██████████ ■ 5. Generate Execution Plan ■
██████████ ■ ▼
██████████ ■ 6. Show Plan to Operator ■
██████████ ■ ▼
██████████ ■ 7. Execute Actions
(create/update/destroy)■ ██████████ ▼
██████████ ■ 8. Update tfstate with new Known State■
██████████
```

CLI, Providers, DAG, State Engine

2.1 Terraform CLI Architecture

Terraform is a single Go binary that orchestrates everything. Understanding its internal architecture is critical for diagnosing issues, building CI/CD pipelines, and designing enterprise workflows.

TERRAFORM BINARY ARCHITECTURE



The Command Lifecycle

Every Terraform workflow follows the same fundamental lifecycle. Understanding each phase deeply prevents common mistakes.

Command	Phase	What It Does	Side Effects	Safe to Run?
terraform init	Initialization	Downloads providers, configures backend, initializes modules	Creates new state file, modifies .tf files	■ Always safe
terraform validate	Validation	Checks HCL syntax and basic schema validation	None	■ Always safe
terraform fmt	Formatting	Rewrites .tf files to canonical HCL style	Modifies .tf files	■ Always safe
terraform plan	Planning	Computes diff between desired state and current state	Reads state file, shows what will be changed	■ Safe to run, but not idempotent
terraform apply	Execution	Executes the plan — creates, updates, or destroys resources	Writes to state file, modifies infrastructure	■ Always safe
terraform destroy	Teardown	Plans and applies destruction of resources	Deletes resources, removes state file	■ Always safe
terraform import	Migration	Associates existing cloud resource with Terraform state	Writes to state file	■ State mutation
terraform refresh	Sync	Updates state file to match actual infrastructure	Writes to state file (deprecated)	■ Avoid, use plan -refresh-only
terraform output	Read	Reads output values from state	None	■ Always safe
terraform graph	Visualization	Outputs DOT graph of resource dependencies	None	■ Always safe

2.2 Provider Plugin System

Providers are the bridge between Terraform and cloud/service APIs. They are separate Go binaries that Terraform downloads during `terraform init` and communicates with via gRPC over a local socket.

PROVIDER PLUGIN LIFECYCLE

[illegible]

The provider configuration block specifies which providers your code requires:

2.3 Resource Dependency Graph (DAG)

Terraform builds a Directed Acyclic Graph (DAG) of all resources before executing any changes. This graph drives parallel execution, dependency ordering, and cycle detection. This is one of Terraform's most powerful architectural features.

[illegible]

The graph building algorithm uses **topological sorting** (Kahn's algorithm) to determine a valid execution order.

The key insight:

- Resources with **no dependencies** execute in the first wave (parallel)
- Resources whose **all dependencies are satisfied** move to the next wave
- **Cycle detection** is performed during graph construction — a cycle causes an immediate error
- The `depends_on` meta-argument adds explicit edges when implicit dependencies don't exist

State Reconciliation, Addressing, Drift Detection

3.1 Resource Addressing & Identity

Resource addressing is the system by which Terraform uniquely identifies every resource it manages. Mastering this is essential for state manipulation, module refactoring, and troubleshooting.

RESOURCE ADDRESS ANATOMY

```
module.network.module.subnets["us-east-1a"].aws_subnet.main
Resource Name Resource Type Module Instance Key (for_each)
Nested Module Name Module Name 'module' keyword Root-level resource:
aws_instance.web Root with count: aws_instance.web[0] Root with for_each:
aws_instance.web["frontend"] Module resource: module.vpc.aws_vpc.main Nested module:
module.infra.module.vpc.aws_vpc.main Deep nested for_each:
module.env["prod"].aws_s3_bucket.data
```

The resource address is the primary key Terraform uses to look up resources in the state file. Each resource address maps to a state entry containing the resource's real cloud ID:

```
# State entry structure (simplified from terraform.tfstate)
{
  "resources": [
    {
      "module": "module.network",
      "mode": "managed",
      "type": "aws_vpc",
      "name": "main",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 1,
          "attributes": {
            "id": "vpc-0a1b2c3d4e5f67890", # ← Cloud resource ID
            "cidr_block": "10.0.0.0/16",
            "enable_dns_hostnames": true,
            "tags": {"Name": "main", "Environment": "prod"}
          }
        }
      ]
    }
  ]
}
```

3.2 What Happens When a Resource is Renamed

This is one of the most common Terraform mistakes. Understanding why Terraform reacts to renames with destroy/create — and how to prevent it — is critical for production operations.

THE RENAME PROBLEM

```

██████████████████████████████████████████████████████████████████████████ BEFORE
(in state): aws_instance.web → i-0abc123 AFTER (in .tf): aws_instance.frontend → ???
Terraform sees: - aws_instance.web → exists in state BUT NOT in code → DESTROY -
aws_instance.frontend → exists in code BUT NOT in state → CREATE Result: DESTROY old
instance + CREATE new instance (downtime, new IP, new ID, data loss risk!) SOLUTION –
moved block: ██████████████████████████████████████████████████████████████████████████ moved {
████ from = aws_instance.web █████ to = aws_instance.frontend █████ } █
██████████████████████████████████████████████████████████████████████████ Terraform now: - Renames
the state entry (no destroy/create) - Zero infrastructure changes

```

moved Blocks — Complete Guide

The moved block (introduced in Terraform 1.1) is the safe, declarative way to handle resource renaming and module reorganization without destroying and recreating resources.

```
# ■■■ Example 1: Simple resource rename ████████████████████████████████████
moved {
    from = aws_instance.web
    to   = aws_instance.frontend
}

# ■■■ Example 2: Moving into a module ████████████████████████████████████████
moved {
    from = aws_vpc.main
    to   = module.networking.aws_vpc.main
}

# ■■■ Example 3: Moving resources when adopting for_each ██████████
# Before: resource "aws_s3_bucket" "logs" {}
# After:  resource "aws_s3_bucket" "logs" { for_each = toset(["access", "app"]) }
moved {
    from = aws_s3_bucket.logs
    to   = aws_s3_bucket.logs["access"]
}

# ■■■ Example 4: Moving between module instances ████████████████████████
moved {
    from = module.servers[0]
    to   = module.servers["web"]
```

}

■ moved blocks are **permanent declarations**. Keep them in your codebase so that anyone who hasn't applied yet also gets the safe rename. They can be removed once everyone has applied, but it's often safer to keep them.

3.3 State Drift Detection

State drift occurs when the actual cloud infrastructure diverges from Terraform's state file. This can happen due to manual changes, external automation, or cloud provider events.

DRIFT DETECTION FLOW

```
Scenario:
Engineer manually changes EC2 instance type in AWS Console from t3.medium → t3.large
Terraform State: { instance_type: "t3.medium" } AWS Reality: { instance_type: "t3.large"
} ← DRIFT
terraform plan (with default -refresh=true)
1. Read state → t3.medium
2. Query AWS API → t3.large (DRIFT FOUND!)
3. Diff: desired=t3.medium, actual=t3.large
4. Plan: ~ aws_instance.web (instance_type) t3.large → t3.medium

DETECT ONLY (no changes): terraform plan -refresh-only → Shows drift without proposing code changes →
Can -apply to ACCEPT the drift into state IGNORE DRIFT for specific attributes: lifecycle
{ ignore_changes = [instance_type] }
```

Three strategies for handling drift:

Strategy	When to Use	Command	Risk
Reconcile to code	Code is the source of truth (standard)	terraform apply	Changes live infra back to code
Accept drift into state	Manual change was intentional	terraform apply -refresh-only	Code and state diverge
Ignore attribute	External system controls the value	ignore_changes = [attr]	Ongoing drift accumulation
Import & codify	Manual resource should be Terraform managed	terraform import → update code	Initial complexity

Local State, Remote Backends, Locking, Corruption Recovery

4.1 Understanding the State File

The Terraform state file (`terraform.tfstate`) is the single most important artifact in a Terraform deployment. It is a JSON file that maps Terraform resource addresses to real cloud resource IDs and their last-known attribute values.

- The state file is the source of truth for Terraform. Losing it without a backup means Terraform loses track of all managed resources — they become orphans that continue to run and accrue cost, but Terraform can no longer manage or destroy them.

STATE FILE ROLE IN THE TERRAFORM WORKFLOW

[illegible]

4.2 Remote State Backends

Local state is dangerous for teams. Remote state backends provide storage, locking, versioning, and encryption. Enterprise environments should always use a remote backend.

AWS Backend (S3 + DynamoDB):

Azure Backend (Blob Storage):

```
terraform {
```

GCP Backend (GCS):

4.3 State Locking — Deep Dive

[illegible]

Terraform from Zero to Mastery — Enterprise IaC Guide

4.4 State Corruption Recovery

State file corruption is a critical incident. Here is the recovery playbook:

```
STATE CORRUPTION RECOVERY PLAYBOOK ██████████ ■ STEP 1:  
Identify corruption (error on terraform plan/apply)■  
██████████ ■ ▼  
██████████ ■ STEP 2: DO NOT run terraform apply (risk  
of orphaning resources)■ ██████████ ■ ▼  
██████████ ■ STEP 3: Pull current state: terraform  
state pull > corrupted.tfstate■ ██████████ ■ ▼  
██████████ ■ STEP 4: Check S3 versioning for previous  
good version■ ██████████ ■ ▼  
██████████ ■ STEP 5: Restore: aws s3api restore-object  
... (version ID)■ ██████████ ■ ▼  
██████████ ■ STEP 6: terraform state pull to verify  
restoration■ ██████████ ■ ▼  
██████████ ■ STEP 7: Run terraform plan to verify no  
unexpected changes■ ██████████ ■ ▼  
██████████ ■ STEP 8: Document incident and review  
locking configuration■ ██████████
```

■■ Always enable S3 versioning on your state bucket and set a lifecycle policy to retain at least 90 days of state versions. This is your primary disaster recovery mechanism.

Every Command, Flag, and Use Case

■■■■ 5.1 Initialization Commands

```
# Standard initialization – downloads providers and modules
terraform init

# Upgrade all provider and module versions to latest matching constraints
terraform init -upgrade

# Reconfigure backend (e.g., migrating from local to S3)
terraform init -reconfigure

# Migrate state to new backend
terraform init -migrate-state

# Initialize without downloading modules (uses cached)
terraform init -get=false

# Initialize without backend (useful for generating plans offline)
terraform init -backend=false

# Custom plugin directory (air-gapped environments)
terraform init -plugin-dir=/opt/terraform/plugins
```

■■■■ 5.2 Planning Commands

```
# Standard plan – shows what will change
terraform plan

# Save plan to file (recommended for CI/CD)
terraform plan -out=tfplan
terraform apply tfplan # Apply exactly what was planned

# Detect drift without proposing changes
terraform plan -refresh-only

# See what destroy would do (without doing it)
terraform plan -destroy

# Limit plan to specific resource(s)
terraform plan -target=aws_instance.web
terraform plan -target=module.networking

# Set variables inline
terraform plan -var="environment=prod" -var="region=us-east-1"
```

```
# Use variables file
terraform plan -var-file="prod.tfvars"

# Skip state refresh (faster but may miss drift)
terraform plan -refresh=false

# Control parallel operations
terraform plan -parallelism=20

# Show plan in JSON format (for tooling)
terraform show -json tfplan | jq .
```

■■■ 5.3 State Sub-Commands — Complete Reference

```
# List all resources in state
terraform state list
terraform state list module.networking      # Filter by module
terraform state list aws_instance.*         # Filter by type

# Show detailed attributes of a resource
terraform state show aws_instance.web
terraform state show module.vpc.aws_vpc.main

# Move resource in state (rename, reorganize)
terraform state mv aws_instance.web aws_instance.frontend
terraform state mv aws_vpc.main module.network.aws_vpc.main

# Remove resource from state WITHOUT deleting it in cloud
# (orphan it – Terraform forgets about it)
terraform state rm aws_instance.old_server
terraform state rm module.legacy           # Remove entire module

# Pull state to stdout (inspect/backup)
terraform state pull > backup-$(date +%Y%m%d).tfstate

# Push local state to remote backend (use with extreme caution!)
terraform state push terraform.tfstate

# Replace provider in state (when provider source changes)
terraform state replace-provider \
  registry.terraform.io/hashicorp/aws \
  registry.terraform.io/hashicorp/aws
```

5.4 Import Commands

Importing existing cloud resources into Terraform management is a common enterprise task, especially during migrations from manual/CloudFormation infrastructure.

■ The **import block + -generate-config-out** workflow is the recommended approach for migrating large existing infrastructures. It generates the HCL code automatically, reducing manual effort dramatically.

Git-Based, State-Based, Blue-Green, Canary

6.1 Why Terraform Has No Native Rollback

This surprises many engineers. Terraform deliberately does not have a built-in rollback command. Understanding why reveals important design philosophy:

- **State is additive:** Terraform applies changes incrementally. A partial failure means some resources are new and some are old. A 'rollback' would need to know which changes to undo.
- **Infra is not a transaction:** Unlike database transactions, infrastructure changes cannot be atomically committed or rolled back. Creating 50 resources is 50 separate API calls.
- **Rollback is a forward operation:** In immutable infrastructure philosophy, the correct response to a failure is to deploy a known-good version forward, not to undo changes.

Terraform's philosophy: if a deployment fails, the correct action is to deploy a previously working version of the code, not to try to reverse-engineer what to undo.

PARTIAL FAILURE SCENARIO

```
terraform apply
(V2 deployment) Resource 1: aws_vpc.main → Created Resource 2:
aws_subnet.public → Created Resource 3: aws_security_group.web → Created
Resource 4: aws_instance.web → FAILED Resource 5: aws_lb.main → Skipped
State now reflects: Resources 1-3 at V2, Resources 4-5 unknown
There is no 'undo' – you must go forward
Options: A) Fix the error and re-run terraform apply
B) Revert code to V1 and terraform apply (forward to old version)
C) Manually destroy the partial changes and investigate
```

6.2 Git-Based Rollback (Primary Method)

The standard enterprise rollback procedure uses Git to revert the code and re-apply the previous working version.

```
# SCENARIO: V2 deployment failed, need to rollback to V1

# 1. Check Git log to find the last known-good commit
git log --oneline -10
# e.g.: abc1234 Add ALB and target groups (V2 - FAILED)
#       def5678 Initial EC2 and VPC setup (V1 - GOOD)

# 2. Option A: Revert the commit (creates a new commit)
git revert abc1234
git push origin main
# CI/CD pipeline runs: terraform apply → restores V1 config
```

6.3 Blue-Green Infrastructure Rollback

Blue-green deployments are the gold standard for zero-downtime rollbacks in Terraform. The pattern maintains two identical environments and switches traffic between them.

[illegible]

```
# terraform apply -var="blue_weight=100" -var="green_weight=0"
```

Controlled Destroy, Partial Destroy, Enterprise Playbook

7.1 Why Terraform is Ideal for Decommissioning

Infrastructure decommissioning is often the most neglected phase of the infrastructure lifecycle. Terraform's state file provides a complete, auditable inventory of every resource that must be retired. This makes Terraform not just a provisioning tool, but a powerful retirement orchestration tool.

■ Terraform's state file IS your decommission checklist. Every resource that was ever created via Terraform is listed in state with its cloud ID, configuration, and dependencies. No manual inventory required.

[illegible]

7.2 Enterprise Decommission Playbook

This is the recommended playbook for retiring production infrastructure:

Phase 1: Inventory & Dependencies

- terraform state list > decommission-inventory.txt
- terraform graph | dot -Tpng > dependency-graph.png
- Identify external dependencies (DNS, other state files, etc.)
- Check for cross-state data sources referencing these resources

Phase 2: Data Archival & Compliance

- Identify stateful resources: RDS, EBS, S3, EFS, DynamoDB
- Create final snapshots of all databases (RDS, DynamoDB export)
- Archive S3 data to Glacier or secondary storage
- Document data retention requirements (GDPR, HIPAA, SOX)
- Verify backup restoration works before destroying source

Phase 3: Traffic Cutover

- Remove DNS records pointing to retiring infrastructure
- Remove from load balancer target groups
- Update service discovery / service mesh configuration
- Verify zero traffic for minimum 24-48 hours

Phase 4: Remove Destroy Protections

- Remove lifecycle { prevent_destroy = true } from all resources
- terraform plan -destroy (review the full destruction plan)
- Get approval from infrastructure owner and security team

Phase 5: Controlled Destruction

- Destroy stateless resources first: EC2, ECS, Lambda
- Destroy load balancers and security groups
- Destroy databases (after final snapshot confirmed)
- Destroy networking (subnets, VPC, IGW, route tables)
- Destroy IAM roles and policies last

Phase 6: Validation

- terraform state list (should return empty or target resources only)
- Verify AWS/Azure/GCP console shows no remaining resources
- Validate cost dashboard shows expected reduction
- Confirm no S3/storage charges accumulating

Phase 7: Audit Evidence

- Archive final state file (even the empty one)
- Archive Git history of decommission changes
- Archive cloud provider billing reports pre/post
- Document in runbook/ITSM ticket with all approvals

■■■■ 7.3 prevent_destroy — Production Safeguard

```
# Prevent accidental destruction of critical resources
resource "aws_db_instance" "production" {
  identifier      = "prod-postgres"
```

```
engine          = "postgres"
instance_class   = "db.r6g.xlarge"
allocated_storage = 500

lifecycle {
  # terraform destroy or plan -destroy will FAIL with this set
  prevent_destroy = true

  # Also ignore auto-generated password rotations
  ignore_changes = [password]
}

# To decommission: first remove prevent_destroy = true
# then commit, review plan, get approval, then terraform destroy
```

`force_destroy`, `apply -replace`, Data Loss Implications

8.1 force_destroy for AWS S3

By default, Terraform cannot delete an S3 bucket that contains objects. `force_destroy = true` instructs Terraform to delete all objects (including all versions if versioning is enabled) before destroying the bucket.

■ `force_destroy = true` on an S3 bucket will permanently delete ALL objects and ALL versions. This is irreversible. Enable only when you are certain the data has been archived or is disposable.

8.2 terraform apply -replace

The `-replace` flag (Terraform 0.15.2+) forces a specific resource to be destroyed and recreated, even if its configuration hasn't changed. This replaces the older `taint` command.

```
# Use cases for -replace:
# • EC2 instance has become unhealthy/corrupted
# • Need to rotate SSH keys by recreating instance
# • Provider has a bug requiring resource recreation
# • After AMI upgrade to force new instance from new AMI
# • Certificate or secret rotation requiring new resource

# Always plan first to see impact
terraform plan -replace=aws_instance.web

# Old way (deprecated, avoid):
# terraform taint aws_instance.web
# terraform apply
```

mv, rm, import, moved blocks, Import blocks

■■■ 9.1 terraform state mv — When and How

State move is used when reorganizing your Terraform code without changing the underlying infrastructure. Common scenarios: module refactoring, renaming resources, adopting for_each.

```
# ■■■ Scenario 1: Moving resource into a module ■■■■■■■■■■■■■■■■■■■■■■
# Before: aws_vpc.main is at root level
# After:   Moving to module.networking
terraform state mv aws_vpc.main module.networking.aws_vpc.main

# ■■■ Scenario 2: Renaming resource ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
terraform state mv aws_s3_bucket.data aws_s3_bucket.raw_data

# ■■■ Scenario 3: Converting count to for_each ■■■■■■■■■■■■■■■■■■■■■■
terraform state mv "aws_instance.servers[0]" "aws_instance.servers[\\"web\\"]"
terraform state mv "aws_instance.servers[1]" "aws_instance.servers[\\"api\\"]"

# ■■■ Verify the move worked ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
terraform state list
terraform plan # Should show no changes if move was correct
```

■ terraform state mv is the OLD way. For Terraform 1.1+, use moved {} blocks instead. They are version-controlled, reviewable in PRs, and self-documenting.

9.2 Import Blocks — Terraform 1.5+

Import blocks are the modern way to bring existing infrastructure under Terraform management. They are declarative, reviewable, and can be used with config generation.

[illegible]

data vs resource, Validation, Cross-Module Communication

■■■■ 10.1 Data Sources vs Resources

This distinction is fundamental. A resource block creates and manages infrastructure. A data block reads existing infrastructure (or external data) without modifying it.

```
# RESOURCE: Terraform creates and owns this
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  # Terraform will CREATE this VPC on apply
  # Terraform will UPDATE it if configuration changes
  # Terraform will DESTROY it on terraform destroy
}

# DATA SOURCE: Terraform reads but does NOT own this
data "aws_vpc" "existing" {
  id = "vpc-0a1b2c3d4e5f67890"
  # Terraform will READ this VPC attributes
  # Terraform will NEVER modify or destroy it
}

# Common data sources:
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-*-22.04-amd64-server-*"]
  }
}

# Read outputs from another Terraform state (cross-state reference)
data "terraform_remote_state" "networking" {
  backend = "s3"
  config = {
    bucket = "my-terraform-state"
    key    = "prod/networking/terraform.tfstate"
    region = "us-east-1"
  }
}

# Use the remote state output
resource "aws_instance" "app" {
  subnet_id = data.terraform_remote_state.networking.outputs.private_subnet_id
}
```

■■■■ 10.2 Variable Types & Validation

```
variable "environment" {
  type      = string
  description = "Deployment environment (dev/staging/prod)"
  default    = "dev"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be one of: dev, staging, prod."
  }
}

variable "instance_count" {
  type      = number
  default    = 2
  validation {
    condition     = var.instance_count >= 1 && var.instance_count <= 100
    error_message = "Instance count must be between 1 and 100."
  }
}

# Sensitive variable – never shown in plan/apply output
variable "db_password" {
  type      = string
  sensitive = true
  # No default – must be provided via TF_VAR_db_password env var
  # or -var="db_password=..." or Vault/Secrets Manager
}

# Complex type: object
variable "tags" {
  type = object({
    team      = string
    cost_center = string
    project    = string
  })
  default = {
    team      = "platform"
    cost_center = "engineering"
    project    = "core-infra"
  }
}
```

`create_before_destroy, ignore_changes, replace_triggered_by`

11.1 Complete Lifecycle Reference

```
resource "aws_db_instance" "main" {
  identifier      = "prod-db"
  engine          = "postgres"
  instance_class  = "db.r6g.xlarge"
  allocated_storage = 100
  username        = "admin"
  password        = var.db_password

  lifecycle {
    # create_before_destroy
    # For zero-downtime replacements:
    # Normal order:  destroy old → create new (downtime!)
    # With this:    create new → verify → destroy old
    create_before_destroy = true

    # prevent_destroy
    # Terraform will ERROR on any destroy operation
    # Protects against accidental terraform destroy
    prevent_destroy = true

    # ignore_changes
    # Tell Terraform to ignore drift in specific attributes
    # Useful when external systems modify these values
    ignore_changes = [
      password,          # Managed by Secrets rotation
      snapshot_identifier, # Set by restore operations
      tags["LastModified"], # Set by external tagging tool
    ]

    # replace_triggered_by
    # Force replacement when referenced resource changes
    # Even if THIS resource configuration is unchanged
    replace_triggered_by = [
      aws_db_subnet_group.main.id, # Recreate if subnet changes
    ]
  }
}
```

Lifecycle Option	Use Case	Risk Level	Example Scenario
create_before_destroy	Zero-downtime resource replacement	Medium — doubles resource briefly	Secret rotation, DB instance class change

Lifecycle Option	Use Case	Risk Level	Example Scenario
prevent_destroy	Production safeguard for critical resources	Low	Prevents accidental deletion of RDS, S3 data buckets
ignore_changes	External system manages specific attributes	Medium	Drift accumulation; ASG desired_count managed by autoscaling
replace_triggered_by	Force replacement on dependency changes	High	Destroying the resource and recreating instances when user_data changes

Design, Versioning, Private Registries, Monorepo vs Multi-Repo

12.1 Module Design Principles

Modules are the primary abstraction mechanism in Terraform. A well-designed module encapsulates related resources and exposes a clean interface via variables and outputs.

[illegible]

12.2 Monorepo vs Multi-Repo

Factor	Monorepo	Multi-Repo
Atomic changes	■ Change VPC + app in one PR	■ Multiple PRs required
Blast radius	■■ Anyone can change anything	■ Isolated per team/service
Versioning	Implicit (git SHA)	■ Explicit semver tags
Dependency tracking	■ Easier (co-located)	■ Must pin module versions
Team autonomy	■ Shared ownership conflicts	■ Clear ownership
CI/CD complexity	Medium (path-based triggers)	Higher (multiple pipelines)
Recommended for	Small-medium teams (<10 engineers)	Large orgs, platform teams

Environments, Workspaces, GitOps, Directory Structure

13.1 Recommended Directory Structure

```
infrastructure/
├── modules/                                # Shared internal modules
│   ├── aws-eks-cluster/
│   ├── aws-rds-postgres/
│   ├── aws-networking/
│   └──
├── environments/
│   ├── dev/
│   │   ├── us-east-1/
│   │   │   ├── networking/
│   │   │   │   ├── main.tf
│   │   │   │   ├── variables.tf
│   │   │   │   └── backend.tf
│   │   │   └── eks/
│   │   │       ├── main.tf
│   │   │       └── backend.tf
│   │   └── staging/
│   │       ├── us-east-1/
│   │       └── prod/
│   │           ├── us-east-1/
│   │           └── eu-west-1/                # Multi-region prod
│   └──
├── .github/
│   ├── workflows/
│   │   ├── terraform-plan.yml             # PR: plan + post results
│   │   └── terraform-apply.yml           # Main: auto-apply on merge
│   └──
├── scripts/
│   ├── bootstrap-backend.sh               # Create S3+DynamoDB
│   └── validate-all.sh                   # Validate all modules
```

13.2 GitOps Integration Patterns

GitOps applies Git-based workflows to infrastructure: pull requests for review, main branch as source of truth, automated apply on merge.

Tool	Type	Key Feature	Best For
GitHub Actions	CI/CD	Native GitHub integration, free tier	GitHub-based orgs
GitLab CI	CI/CD	Built-in, no external tools needed	GitLab-based orgs

Tool	Type	Key Feature	Best For
Atlantis	Terraform-specific	PR-based plan/apply comments	Teams wanting PR workflow
Spacelift	SaaS IaC platform	Policy, drift detection, audit	Enterprise governance
Terraform Cloud	HashiCorp SaaS	Remote state, Sentinel policy	HashiCorp ecosystem
Azure DevOps	CI/CD	Deep Azure integration	Microsoft/Azure shops
Jenkins	CI/CD	Highly customizable, self-hosted	Existing Jenkins orgs

Example GitHub Actions workflow for Terraform:

```
# .github/workflows/terraform.yml
name: Terraform
on:
  pull_request:
    paths: ["environments/**", "modules/**"]
  push:
    branches: [main]
    paths: ["environments/**", "modules/**"]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.9.0"

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/TerraformCI
          aws-region: us-east-1

      - name: Terraform Init
        run: terraform init

      - name: Terraform Plan
        run: terraform plan -out=tfplan -no-color
        if: github.event_name == "pull_request"

      - name: Post Plan to PR
        uses: actions/github-script@v7
        if: github.event_name == "pull_request"
        # ... (post plan output as PR comment)

      - name: Terraform Apply
```

```
run: terraform apply -auto-approve
if: github.ref == "refs/heads/main"
```

Secrets, IAM, State Security, SAST Scanning

■■■ 14.1 Secret Management — Never in Code

■ NEVER hardcode secrets in .tf files or .tfvars files. They end up in Git history and in the state file (which may be stored in S3 without encryption).

```
# ■ WRONG – credential in code
resource "aws_db_instance" "main" {
  password = "SuperSecret123!" # This will be in Git AND state!
}

# ■ CORRECT – use AWS Secrets Manager
data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = "prod/database/master-password"
}

resource "aws_db_instance" "main" {
  password = jsondecode(data.aws_secretsmanager_secret_version.db_password.secret_string)["password"]

  lifecycle {
    ignore_changes = [password] # Allow external rotation
  }
}

# ■ CORRECT – use HashiCorp Vault
data "vault_generic_secret" "db_creds" {
  path = "secret/prod/database"
}

resource "aws_db_instance" "main" {
  username = data.vault_generic_secret.db_creds.data["username"]
  password = data.vault_generic_secret.db_creds.data["password"]
}
```

■■■ 14.2 IAM Least Privilege for Terraform

The Terraform CI/CD service role should have only the permissions required — no more. Use condition keys to restrict to specific resources and regions.

```
# Terraform CI role – restrict to specific regions and environments
{
  "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Sid": "TerraformStateAccess",
    "Effect": "Allow",
    "Action": [
      "s3:GetObject", "s3:PutObject", "s3:ListBucket"
    ],
    "Resource": [
      "arn:aws:s3::my-company-terraform-state",
      "arn:aws:s3::my-company-terraform-state/*"
    ]
  },
  {
    "Sid": "TerraformLocking",
    "Effect": "Allow",
    "Action": [
      "dynamodb:GetItem", "dynamodb:PutItem", "dynamodb:DeleteItem"
    ],
    "Resource": "arn:aws:dynamodb:us-east-1:123456:table/terraform-locks"
  }
]
// Add only the specific EC2/RDS/VPC permissions needed
}

```

■ ■ Use **OIDC authentication** with GitHub Actions / GitLab CI to assume IAM roles without storing long-lived AWS credentials as secrets. This is the recommended pattern for CI/CD.

17 Critical Terraform Anti-Patterns & Remediation

15.0 The 17 Critical Terraform Anti-Patterns

AP-01 Monolithic State File

Problem: All environments (dev/staging/prod) in a single state file. A failed prod apply can corrupt all environments.

Fix: Split state by environment AND by layer (networking/compute/data).

AP-02 Local State in Teams

Problem: Using local terraform.tfstate on a developer's laptop. No locking, no sharing, high risk of overwrites.

Fix: Always use remote backends (S3+DynamoDB, Terraform Cloud, GCS) in team settings.

AP-03 Manual Cloud Changes

Problem: Engineers make console/CLI changes that bypass Terraform. State drift accumulates silently.

Fix: Enforce IaC-only changes via IAM SCPs, Azure Policies, or org policies. Use drift detection CI jobs.

AP-04 Hardcoded Values

Problem: `account_id = "123456789012"` in code. Makes module reuse impossible and causes security issues.

Fix: Use variables, data sources, and locals. Pass account IDs via variables or `data.aws_caller_identity`.

AP-05 Secrets in Plaintext

Problem: `password = "Admin123!"` in .tf files. Shows in plan output AND state file. Git history exposure.

Fix: Use Secrets Manager, Key Vault, or HashiCorp Vault. Mark variables as `sensitive = true`.

AP-06 No State Versioning/Backup

Problem: S3 bucket without versioning. One accidental state push can permanently destroy state.

Fix: Enable S3 versioning + MFA delete. Automate state backups to separate bucket.

AP-07 Excessive -target Usage

Problem: Using `-target` to deploy partial changes routinely. Creates state inconsistencies over time.

Fix: `-target` is for emergencies only. Fix the underlying issue instead of targeting around it.

AP-08 Ignoring .terraform.lock.hcl

Problem: Not committing the lock file. Different engineers use different provider versions.

Fix: Commit `.terraform.lock.hcl` to Git. Run terraform providers lock for multi-platform support.

AP-09**Unpinned Provider Versions**

Problem: version = ">= 2.0" allows major version upgrades breaking your code silently.

Fix: Use ~> (pessimistic constraint): version = "~> 5.0" allows 5.x but blocks 6.x.

AP-10**Shared Prod Credentials in CI**

Problem: Storing long-lived IAM access keys in CI secrets. If leaked, attacker has full infra access.

Fix: Use OIDC role assumption for GitHub Actions / GitLab CI. No long-lived credentials needed.

AP-11**No destroy Protection**

Problem: Production databases without prevent_destroy. A mistyped terraform destroy destroys prod data.

Fix: Add lifecycle { prevent_destroy = true } to all stateful production resources.

AP-12**Workspace Abuse for Environments**

Problem: Using terraform workspace new prod/dev for environment separation. State shares same backend key prefix.

Fix: Use separate state files per environment in separate directories, not workspaces.

AP-13**Giant Modules**

Problem: 500+ line monolithic modules that do everything. Impossible to test, reason about, or reuse.

Fix: Single responsibility: one module per logical component. Max ~200 lines per module.

AP-14**No Terraform Version Pinning**

Problem: No required_version constraint. Different engineers run different Terraform versions.

Fix: Always set required_version = "~> 1.9" and enforce with tfenv or mise.

AP-15**apply Without Reviewing Plan**

Problem: Using -auto-approve in production without reviewing the plan. Surprises await.

Fix: Always run plan first. Use -auto-approve only in CI after plan is reviewed in PR.

AP-16**No Terraform Formatting CI Check**

Problem: Inconsistent formatting causes noisy PRs and makes code reviews harder.

Fix: Add terraform fmt -check to CI pipeline. Fail PR if formatting is not canonical.

AP-17**Importing Without Understanding**

Problem: Running terraform import and then terraform apply without reviewing the generated plan first.

Fix: Always run terraform plan after import to understand what Terraform wants to change.

State Locks, Drift, Import Failures, Provider Issues

■■■ 16.1 Error Resolution Quick Reference

■ Error acquiring the state lock

- Run: `terraform force-unlock` (find ID in error message)
- Verify no other terraform process is running first!
- Check DynamoDB table for stale lock entries
- If DynamoDB lock is stuck: `aws dynamodb delete-item --table-name terraform-locks --key '{"LockID":{"S":""}}'`

■ Error: Resource already exists

- Resource exists in cloud but not in Terraform state
- Option A: Import it: `terraform import`
- Option B: Rename your resource to not conflict
- Option C: Delete the cloud resource if it's not needed

■ Error: Provider version conflict

- Run: `terraform init -upgrade`
- Check `required_providers` version constraints — may be too restrictive
- Delete `.terraform.lock.hcl` and re-run `terraform init`
- Check for conflicting version requirements between modules

■ Inconsistent dependency lock file

- Delete `.terraform.lock.hcl`
- Run: `terraform init`
- Commit the regenerated lock file
- Ensure all team members run: `terraform providers lock -platform=linux_amd64 -platform=darwin_arm64`

■ Error: State file locked by unknown process

- Verify no running terraform processes: `ps aux | grep terraform`
- Check CI/CD for running jobs
- Safely force unlock: `terraform force-unlock`
- Review who triggered the lock in DynamoDB Info column

■ Plan shows unexpected resource replacements

- Check if resource has `create_before_destroy` conflicts

- Look for attribute changes that force replacement (ForceNew in provider docs)
- Review moved {} blocks for missing migrations
- Run: terraform state show to compare with actual config

■ terraform destroy fails on S3 bucket

- Bucket has objects — either empty manually or add force_destroy = true
- If versioned: must delete all versions: aws s3 rm s3://bucket --recursive then versioned objects
- Check for bucket policies blocking deletion
- If in CI: script to empty bucket before terraform destroy

■ Import fails with 'no resource with address found'

- Ensure the resource block EXISTS in your .tf files before running import
- Check exact resource address: terraform state list (after import attempt)
- For modules: terraform import module.name.resource_type.name
- Use import {} blocks (Terraform 1.5+) instead of CLI import for reliability

Data Center Exits, Cloud Cleanup, M&A; Integration, Regulatory Decommissioning

17.1 Terraform as a Retirement Orchestration Tool

Large organizations regularly face infrastructure retirement challenges: data center exits, end-of-life application sunsets, cloud migration cleanup, merger and acquisition integration, and regulatory decommissioning. Terraform's state file is uniquely valuable in all of these scenarios.

Retirement Scenario	Terraform's Role	Key Terraform Features Used
Data Center Exit	Inventory all managed resources, orderly tear-down by type	state list, plan, destroy, prevent_destroy removal
End-of-Life Application	Map all resources to application, controlled staged destruction	tagged-based filtering, module destroy, state inspection
Cloud Migration Cleanup	Retire old VMs/DBs after workload moved to cloud	import (to get old into state), then destroy
Post-M&A Integration	Inventory acquired infra, unify under corporate Terraform	terraform import bulk, state mv, module consolidation
Cost Optimization	Identify and destroy unused resources from state	state list, cloud cost tagging, targeted destroy
Regulatory (GDPR/HIPAA)	Documented, auditable destruction with evidence	plan -destroy review, Git history, state backup audit
Kubernetes Cluster Retirement	Drain workloads, delete cluster and all nodes	kubectl drain → terraform destroy module.eks
DNS Zone Retirement	Remove DNS records in correct dependency order	plan -destroy shows record → zone ordering

17.2 Bulk Resource Retirement Pattern

```
# Step 1: Audit everything in state
terraform state list > full-inventory-$(date +%Y%m%d).txt

# Count resources by type
terraform state list | sed "s/\[.*\]//" | cut -d. -f1 | sort | uniq -c | sort -rn

# Step 2: Preview full destruction
terraform plan -destroy -out=destroy-plan.tfplan

# Review the plan in JSON (for compliance review)
terraform show -json destroy-plan.tfplan | jq '.resource_changes[] | {address: .address, action: .change.action}'

# Step 3: Remove all prevent_destroy protections
# Search for prevent_destroy = true across all files
grep -r "prevent_destroy" . --include="*.tf"

# Step 4: Staged destruction (safest approach)
```


Licensing, Migration, AI-Assisted IaC, Trends

18.1 HashiCorp Licensing Change & OpenTofu

In August 2023, HashiCorp changed Terraform's license from Mozilla Public License 2.0 (MPL-2.0, open-source) to the Business Source License 1.1 (BUSL-1.1), which restricts competitive commercial use. This was a significant shift in the open-source ecosystem.

TERRAFORM vs OPENTOFU TIMELINE

Aug 2023 ■ HashiCorp changes Terraform to BUSL-1.1 license ■ Key restriction: cannot use to build competing products Aug 2023 ■ OpenTofu fork announced (Linux Foundation) ■ Supported by: Gruntwork, Spacelift, env0, Scalr, ■ Harness, Prefect, Digger, and 100+ other companies Jan 2024 ■ OpenTofu 1.6.0 GA released ■ 100% compatible with Terraform syntax & state format Apr 2024 ■ HashiCorp acquired by IBM (~\$6.4B) 2024+ ■ OpenTofu adds features beyond Terraform: ■ • Provider-defined functions ■ • State encryption (native) ■ • for_each in module blocks ■ • Improved testing framework Today ■ Both are production-viable. ■ OpenTofu: open-source, community-governed ■ Terraform: commercial, HashiCorp/IBM governed

18.2 OpenTofu vs Terraform Feature Comparison

Feature	Terraform	OpenTofu
License	BUSL-1.1 (restricted commercial)	MPL-2.0 (open source)
State format	tfstate JSON	Compatible tfstate JSON
HCL syntax	Terraform HCL	100% compatible HCL
Provider registry	registry.terraform.io	registry.opentofu.org (+ TF compatible)
State encryption	Backend-level only	■ Native built-in encryption
for_each in modules	■ Not supported	■ Supported natively
Provider functions	Limited	■ Provider-defined functions
Test framework	terraform test	■ Enhanced testing
Migration path	Current users	Drop-in replacement (rename binary)
Commercial support	HashiCorp/IBM	Multiple vendors (Spacelift, env0)

Feature	Terraform	OpenTofu
Governance	HashiCorp/IBM	Linux Foundation (community)

18.3 Migration from Terraform to OpenTofu

```
# Migration is a binary swap – no code changes required

# Step 1: Install OpenTofu
# macOS:
brew install opentofu

# Linux:
curl --proto "https" --tls1.2 -fsSL https://get.opentofu.org/install-opentofu.sh | sh

# Step 2: Replace terraform with tofu in your commands
tofu init
tofu plan
tofu apply

# Step 3: Update CI/CD scripts
# Replace: terraform → tofu in all pipeline configs

# Step 4: Update GitHub Actions
# Replace: hashicorp/setup-terraform → opentofu/setup-opentofu

# State format: IDENTICAL – no state migration needed!
# terraform.tfstate works with both tools
```

18.4 Future Trends: AI-Assisted IaC

The convergence of Large Language Models with Infrastructure as Code is creating new paradigms for infrastructure management:

- **Natural Language → Terraform:** Engineers describe desired infrastructure in English; LLMs generate the Terraform code. Human review of plan output remains critical.
- **AI Drift Analysis:** LLMs analyze terraform plan output and explain in plain English what will change and why it might be risky.
- **Policy as Code Generation:** AI generates OPA/Sentinel policies from natural language compliance requirements.

- **Self-Healing Infrastructure:** AI agents detect drift via scheduled plans and automatically create PRs to reconcile, with human approval gates.
- **Cost Optimization AI:** AI analyzes state files and suggests right-sizing, deletion of unused resources, and architecture improvements.

■ ■ AI-generated Terraform code requires rigorous human review. LLMs can generate plausible-looking but incorrect or insecure configurations. Always run terraform plan and review thoroughly before applying AI-generated infrastructure code.

APPENDIX A — CLI CHEAT SHEET

Category	Command	Notes
Init	terraform init	Initialize working directory
Init	terraform init -upgrade	Upgrade providers to latest matching
Init	terraform init -reconfigure	Force backend reconfiguration
Validate	terraform validate	Syntax check only
Format	terraform fmt -recursive	Format all .tf files recursively
Plan	terraform plan	Show execution plan
Plan	terraform plan -out=tfplan	Save plan to file
Plan	terraform plan -destroy	Preview destroy without applying
Plan	terraform plan -refresh-only	Show drift only, no changes proposed
Plan	terraform plan -target=X	Limit plan to resource X (emergency only!)
Apply	terraform apply	Apply with interactive approval
Apply	terraform apply tfplan	Apply a saved plan exactly
Apply	terraform apply -auto-approve	Apply without confirmation (CI only)
Apply	terraform apply -replace=X	Force destroy+recreate resource X
Destroy	terraform destroy	Destroy all managed resources
Destroy	terraform plan -destroy -out=d.tfplan	Plan destroy, save, review, then apply
State	terraform state list	List all resources in state
State	terraform state show X	Show resource X attributes
State	terraform state mv A B	Rename/move resource in state
State	terraform state rm X	Remove from state (orphan resource)
State	terraform state pull > file	Backup state to local file
State	terraform force-unlock ID	Force release stuck lock (DANGEROUS)
Import	terraform import X cloud_id	Import existing resource (legacy)
Import	import {} block + plan -generate-config-out=f.tf	Modern import (TF 1.5+)
Debug	TF_LOG=DEBUG terraform plan	Enable verbose logging
Debug	terraform console	Interactive expression evaluator

Category	Command	Notes
Debug	terraform graph dot -Tpng > g.png	Visualize dependency graph
Output	terraform output	Show all outputs
Output	terraform output -json	Outputs as JSON
Workspace	terraform workspace list	List workspaces
Workspace	terraform workspace new dev	Create new workspace
Workspace	terraform workspace select prod	Switch workspace

APPENDIX B — 125 INTERVIEW QUESTIONS

Beginner Questions (1-50)

- B01.** What is Terraform and how does it differ from configuration management tools like Ansible?
- B02.** What is Infrastructure as Code and why is it important?
- B03.** Explain the difference between terraform plan and terraform apply.
- B04.** What is a Terraform provider? Give 5 examples.
- B05.** What is the terraform.tfstate file and why is it important?
- B06.** What is the difference between resource and data blocks in Terraform?
- B07.** What does terraform init do? What does it create?
- B08.** How do you declare a variable in Terraform? How do you pass a value to it?
- B09.** What is an output value in Terraform and when would you use it?
- B10.** Explain the purpose of the .terraform.lock.hcl file.
- B11.** What is the difference between count and for_each?
- B12.** What are locals in Terraform? Give an example use case.
- B13.** What is terraform fmt and why should you run it?
- B14.** How do you reference an attribute of one resource in another resource?
- B15.** What does terraform destroy do? When would you use it?
- B16.** What is a Terraform module? What are the benefits of using modules?
- B17.** How do you handle sensitive variables in Terraform?
- B18.** What is the purpose of the depends_on meta-argument?
- B19.** How do you use terraform import and when is it needed?
- B20.** What does 'idempotent' mean in the context of Terraform?
- B21.** What is a backend in Terraform? Give examples of remote backends.
- B22.** How do Terraform workspaces work? What are their limitations?
- B23.** What is the lifecycle meta-argument? Name 4 lifecycle options.
- B24.** How do you pass outputs from one module to another?
- B25.** What is the terraform console command used for?
- B26.** What happens if someone makes manual changes to cloud resources managed by Terraform?
- B27.** What is terraform validate and what does it check?
- B28.** How do you specify which version of a provider to use?
- B29.** What is the difference between terraform apply and terraform apply tfplan?
- B30.** What is a Terraform registry?
- B31.** Explain the declarative vs imperative approach with examples.
- B32.** What types are supported for Terraform variables?

- B33.** How do you use a `.tfvars` file?
- B34.** What is the purpose of the `required_version` setting?
- B35.** What happens if you rename a resource block in Terraform?
- B36.** How does `for_each` differ from `count` when managing collections?
- B37.** What is the `terraform graph` command and what does its output represent?
- B38.** What is state locking and why is it important?
- B39.** How do you reference the current AWS account ID in Terraform?
- B40.** What is the `null_resource` / `terraform_data` resource used for?
- B41.** What environment variable enables Terraform debug logging?
- B42.** How do you pass provider configuration from a root module to a child module?
- B43.** What is the difference between a data source and a resource?
- B44.** How do you use string interpolation in Terraform HCL?
- B45.** What are the three main files you'd find in a typical Terraform module?
- B46.** How do you iterate over a map using `for_each`?
- B47.** What is the dynamic block used for?
- B48.** What is `provisioner` in Terraform? Why are they discouraged?
- B49.** What is the `terraform output` command used for?
- B50.** How do you validate variable values using validation blocks?

Advanced Questions (51-100)

- A01.** Explain the Terraform DAG and how it enables parallel execution.
- A02.** How does Terraform handle partial failures during `terraform apply`?
- A03.** What is state drift and how do you detect and remediate it?
- A04.** Explain the difference between `terraform state mv` and `moved` blocks.
- A05.** How does the `moved` block work internally in Terraform state?
- A06.** What are the risks of using `-target` in production?
- A07.** How would you migrate a large infrastructure from manually-provisioned to Terraform-managed?
- A08.** Explain how `terraform plan -refresh-only` differs from `terraform plan`.
- A09.** What is the `create_before_destroy` lifecycle option and when would you use it?
- A10.** How do you implement blue-green deployments in Terraform?
- A11.** Explain cross-state references using `terraform_remote_state`.
- A12.** What is state serialization format? What is the 'serial' field in state?
- A13.** How do you structure Terraform code for multi-environment deployments?
- A14.** What is the difference between provider aliasing and multiple provider configurations?
- A15.** How do you implement zero-downtime database migrations with Terraform?
- A16.** Explain `import` blocks (Terraform 1.5+) vs legacy `terraform import` command.
- A17.** How does Terraform handle resource dependencies with `for_each` collections?

- A18.** What is the `-generate-config-out` flag and how does it work?
- A19.** How would you implement Terraform for a multi-account AWS Organization?
- A20.** Explain the risks of `force_destroy` on an S3 bucket.
- A21.** How do you encrypt Terraform state at rest?
- A22.** What is a provider lock and how do you manage cross-platform locks?
- A23.** How do you test Terraform modules? What tools are available?
- A24.** Explain the difference between `terraform destroy` and using `lifecycle prevent_destroy`.
- A25.** How does Terraform's gRPC communication with providers work?
- A26.** What is a Terraform sentinel policy and how does it enforce governance?
- A27.** How do you handle Terraform state in a monorepo with hundreds of modules?
- A28.** What are the tradeoffs between Terraform workspaces and directory-based environments?
- A29.** How do you safely rotate database passwords managed by Terraform?
- A30.** Explain Terraform's resource lifecycle phases: Create, Read, Update, Delete.
- A31.** How do you implement canary deployments with Terraform?
- A32.** What is the `replace_triggered_by` lifecycle meta-argument?
- A33.** How do you handle provider version constraints in a shared module library?
- A34.** Explain how Terraform resolves type mismatches between variable types.
- A35.** What is the difference between `jsonencode()` and `tostring()` in Terraform?
- A36.** How do you use Terraform to manage Kubernetes manifests?
- A37.** What is the purpose of `terraform providers lock` command?
- A38.** How do you implement Infrastructure as Code for AI/ML platforms?
- A39.** How would you decommission a production environment using Terraform?
- A40.** Explain how `terraform apply -replace` works vs the deprecated `taint` command.
- A41.** How do you handle circular dependencies in Terraform?
- A42.** What is the ephemeral resource type introduced in newer Terraform versions?
- A43.** How do you implement GitOps with Terraform and Atlantis?
- A44.** How would you implement cost governance using Terraform policies?
- A45.** Explain OPA (Open Policy Agent) integration with Terraform plans.
- A46.** How do you manage Terraform provider upgrades in a large codebase?
- A47.** What is the purpose of the `precondition` and `postcondition` lifecycle blocks?
- A48.** How do you implement Terraform testing with the `terraform test` framework?
- A49.** How do you handle state migration when splitting a monolithic state file?
- A50.** What strategies exist for recovering from corrupted Terraform state?

Architect-Level Questions (101-125)

- AR01.** Design a Terraform architecture for a Fortune 500 company with 50+ engineering teams, multiple cloud providers, and strict compliance requirements.

- AR02.** How would you architect Terraform for a company migrating from on-premises data centers to AWS, with a 2-year timeline and 500+ servers?
- AR03.** What is your strategy for managing Terraform state across 200+ microservices, each with dev/staging/prod environments?
- AR04.** Design a Terraform module that implements a complete production-ready EKS cluster with auto-scaling, network policies, and monitoring.
- AR05.** How would you implement a Terraform-based Infrastructure as a Service (IaaS) platform for internal teams, with self-service capabilities?
- AR06.** Describe your approach to Terraform state management for a company going through a merger and acquisition, inheriting infrastructure from the acquired company.
- AR07.** How would you implement a fully automated compliance framework for Terraform, covering SOC 2, HIPAA, and PCI-DSS?
- AR08.** Design a cost optimization strategy using Terraform for a company spending \$2M/month on AWS.
- AR09.** How would you architect Terraform for multi-cloud (AWS + Azure + GCP) with unified governance, consistent tagging, and cross-cloud networking?
- AR10.** What is your disaster recovery architecture for Terraform state, considering state corruption, accidental destroy, and regional failures?
- AR11.** How do you architect Terraform for a CI/CD platform that runs 1000+ pipeline jobs per day, each provisioning temporary testing environments?
- AR12.** Design a Terraform module registry strategy for an enterprise with 300+ engineers, ensuring security, versioning, and discoverability.
- AR13.** How would you implement progressive delivery (canary/blue-green) for infrastructure changes using Terraform in a zero-downtime requirement?
- AR14.** Describe a Terraform strategy for managing multi-tenant SaaS infrastructure where each customer gets isolated cloud resources.
- AR15.** How do you approach Terraform code review as an architect? What automated checks do you require in CI?
- AR16.** Design a drift detection and auto-remediation system using Terraform, AWS Lambda, and EventBridge.
- AR17.** How would you migrate an organization from Terraform Cloud to a self-hosted Atlantis + S3 backend setup?
- AR18.** Describe your approach to Terraform for AI/ML infrastructure (GPU clusters, Databricks, SageMaker, vector databases).
- AR19.** How do you architect Terraform for infrastructure decommissioning programs that span 18+ months and multiple application retirements?
- AR20.** What is your strategy for adopting OpenTofu in an organization that currently uses Terraform Cloud?
- AR21.** Design a Terraform-based Internal Developer Platform (IDP) using Backstage as the frontend and Terraform as the provisioning backend.
- AR22.** How would you implement AI-assisted Terraform code review and security scanning in a GitOps pipeline?
- AR23.** Describe how Terraform integrates with a Service Mesh (Istio/Linkerd) deployment architecture.
- AR24.** How do you design Terraform for regulated environments (financial services, healthcare) with immutable audit trails?
- AR25.** What is your vision for the future of Infrastructure as Code over the next 5 years?

APPENDIX C — PRODUCTION READINESS CHECKLIST

■ State Management

- Remote backend configured (S3+DynamoDB / Azure Blob / GCS)
- State file encryption enabled (at-rest and in-transit)
- State versioning enabled (S3 bucket versioning / GCS object versioning)
- DynamoDB locking table configured (AWS) or native locking (Azure/GCP)
- State backup schedule defined and tested
- State access controls: least-privilege IAM for state bucket
- State file never committed to Git (.gitignore includes *.tfstate)

■ Security

- No secrets/passwords in .tf files or .tfvars
- All sensitive variables marked sensitive = true
- Secrets sourced from Secrets Manager / Key Vault / Vault
- CI/CD uses OIDC role assumption (not long-lived credentials)
- Terraform CI role follows least-privilege IAM
- SAST scanning enabled (Checkov, tfsec, or Snyk IaC)
- Provider version constraints use ~> (pessimistic)
- .terraform.lock.hcl committed to Git

■ Destroy Protection

- lifecycle { prevent_destroy = true } on all production stateful resources
- Production databases have deletion_protection = true
- S3 buckets with critical data do NOT have force_destroy = true
- terraform destroy requires manual approval in CI (not auto-approved)
- Final snapshot before_destroy configured for databases

■ Code Quality

- required_version constraint set in all root modules
- required_providers versions pinned with ~>

- terraform fmt -recursive passes with no changes
- terraform validate passes with no errors
- All resources have consistent tagging (environment, team, cost_center)
- Module README.md documents all inputs, outputs, and examples
- No hardcoded account IDs, region names, or environment values

■ CI/CD Pipeline

- terraform plan runs on every PR and posts results as comment
- terraform apply only runs from main/master branch
- Plan output reviewed before apply is triggered
- Plan saved with -out and same plan applied (not re-planned)
- No -auto-approve in production pipelines without plan review
- Pipeline uses pinned Terraform/OpenTofu version
- Drift detection job runs on schedule (daily minimum)

■ Operations

- Runbook exists for common operations (apply, destroy, import)
- Runbook exists for state corruption recovery
- On-call engineers can manually unlock state
- State is accessible to multiple engineers (not one person's laptop)
- Cost impact reviewed as part of infrastructure changes
- Resource naming conventions documented and enforced

APPENDIX D — DAY-0, DAY-1, DAY-2 OPERATIONS GUIDE

Day-0 Operations: Platform Setup

Day-0 is about establishing the foundation before any application infrastructure is provisioned.

```
# 1. Bootstrap remote state backend (run once by platform team)
cd infrastructure/bootstrap
terraform init -backend=false # Local state for bootstrapping only
terraform apply               # Creates S3 bucket + DynamoDB table

# 2. Set up Terraform version manager
brew install tfenv
tfenv install 1.9.0
tfenv use 1.9.0

# 3. Configure provider credentials
aws configure sso # AWS SSO
az login          # Azure
gcloud auth application-default login # GCP

# 4. Initialize the first environment
cd environments/dev/us-east-1/networking
terraform init
terraform plan # Verify no errors before first apply

# 5. Set up CI/CD OIDC role for GitHub Actions
# (One-time setup per GitHub org)
```

Day-1 Operations: Routine Changes

Day-1 operations are the routine change management activities for maintained infrastructure.

```
# Standard change workflow:
# 1. Create feature branch
git checkout -b feature/add-rds-replica

# 2. Make infrastructure changes
vim environments/prod/us-east-1/data/main.tf

# 3. Validate and format
terraform validate && terraform fmt -recursive

# 4. Run plan locally
```

Day-2 Operations: Maintenance & Incident Response

Day-2 operations cover the ongoing maintenance, incident response, and health monitoring of infrastructure.

[illegible]

End of Terraform from Zero to Mastery Guide

This guide covers Terraform from first principles through enterprise-scale production operations. The most important concepts to internalize are: the state file as source of truth, the declarative reconciliation loop, safe resource lifecycle management via moved blocks and lifecycle controls, Git-based rollback as the primary recovery mechanism, and Terraform state as an auditable inventory for infrastructure decommissioning.

For questions, updates, and community discussion, see:

- <https://developer.hashicorp.com/terraform/docs>
- <https://opentofu.org/docs/>
- <https://registry.terraform.io>
- <https://github.com/opentofu/opentofu>